



**SCHOOL OF
ENGINEERING**

Department of Computer Science Engineering

Major Project Phase-II (Review-1)

**Intellectual Smart Hybrid AI System for Real-Time
Threat Intelligence in the
Internet**

Team No : 89

Under the Supervision

Dr. Naitik S T

Assistant Professor, CSE

Presented By:

NU Chinnappa - ENG22CS0377

Sriram Puranik - ENG22CS0467

Tejas KM - ENG22CS0480

Kalyan Gogoi - ENG23CS1031

Overview

- Abstract
 - Project Scope & Clarity
 - Literature Survey
 - Detailed Design
 - Implementation Approach
 - Demonstration
 - Q&A
- Your paragraph text

Abstract

- Cyber attacks today are becoming more complex and continuously changing.
- Traditional intrusion detection systems mainly detect only known attacks.
- To overcome this limitation, we propose IS-HAITI, a Hybrid AI based Threat Intelligence System.
- The system combines:
 - Graph Neural Network to understand relationships between network services
 - Transformer model to analyze attack behavior over time
 - Rule-based detection for already known threats
- The system supports different inputs such as network traffic data, URLs and text information.
- An explainability module (X-TIS) is used to show why a threat is detected.
- Expected result:
 - Better attack detection
 - Reduced false alarms
- Clear and understandable threat analysis.

Project Scope & Clarity

The proposed project, Intellectual Smart Hybrid AI System for Real-Time Threat Intelligence (IS-HAITI), aims to develop an intelligent cybersecurity system capable of detecting digital threats using Hybrid AI techniques.

The system integrates Transformer models, Graph Neural Networks (GNN), rule-based analysis, and Explainable AI (X-TIS) to analyze communication content and web resources for identifying malicious activities.

Current Implementation

The project currently focuses on:

- A. Spam, Phishing & Scam Detection
 - Detection of spam and phishing messages
 - Identification of fraudulent communications
 - Detection of malicious links in messages
- B. Malicious URL & Website Detection
 - Detection of phishing websites
 - Fake login and credential harvesting pages
 - Suspicious and redirected URLs

Project Scope & Clarity

Project Clarity & Future Scope

The developed system performs content-based and URL-based threat detection and generates explainable outputs describing why a threat is identified as malicious.

Project Boundary

- Offline threat detection and analysis
- Explainable AI-based reporting
- Detection result generation for security analysis

Future Scope

The proposed hybrid architecture can be extended to detect:

- Network-based attacks
- Lateral movement attacks
- Botnet and command-control activities
- Data exfiltration attacks
- Malware behavioral attacks
- Multi-stage and unknown cyber threats
- This enables future expansion toward a complete intelligent threat intelligence platform.

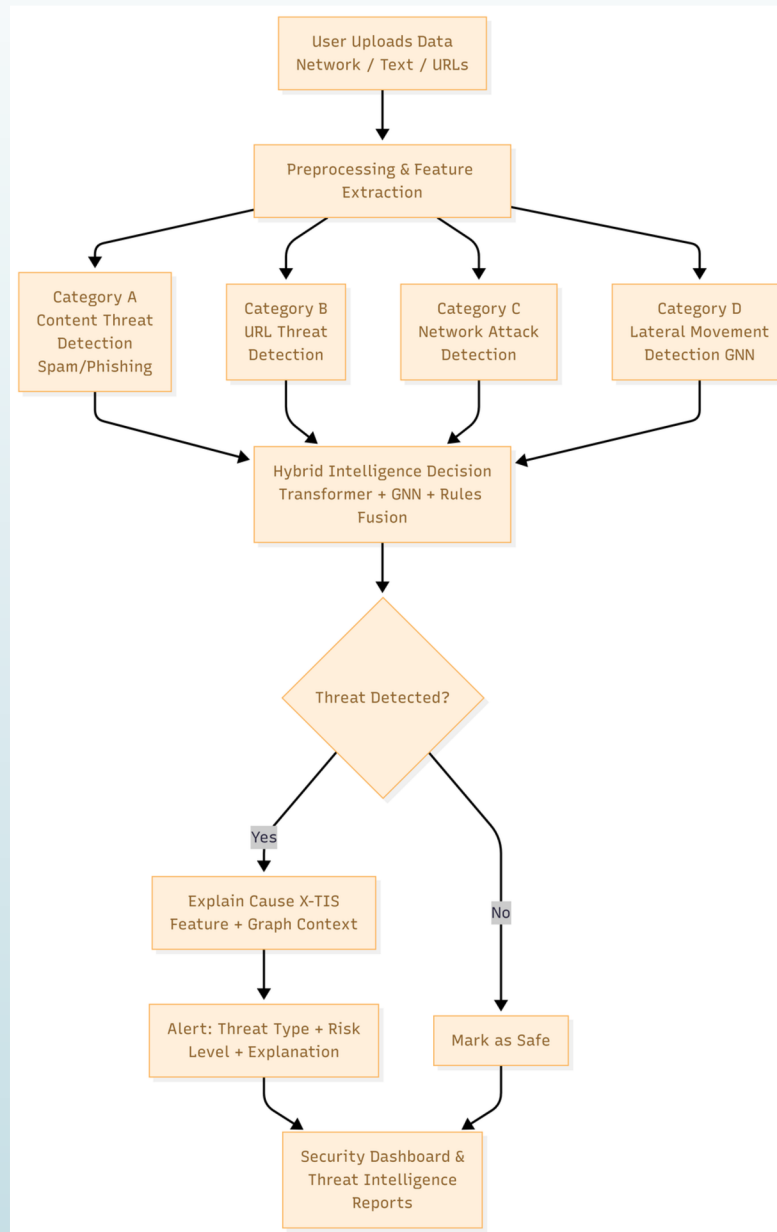
Literature Survey

Author's Name/ Paper Title	Conference/Journal Name and year	Technology/ Design	Results shared by author	What you infer
Kim et al. – AI-Driven Transformer Framework for Network Anomaly Detection	IJACSA, 2025	Transformer-based deep learning for network traffic anomaly detection	Achieved improved detection accuracy by learning long-range behavioral dependencies	Temporal learning is powerful but lacks structural relationship analysis between network entities
Zhou et al. – Graph Neural Networks Embedded with Domain Knowledge for Threat Intelligence	ACM Computing Systems, 2025	Graph Neural Networks (GNN) for relational attack detection	Successfully identified coordinated and lateral movement attacks	Structural modeling improves detection of complex multi-stage cyber attacks
Demir & Koc – DeepGuard Transformer-Based Cyber Threat Detection	International Journal of Computer Engineering, 2025	Transformer-based real-time cyber threat detection	High accuracy in evolving threat environments	Requires explainability and complementary models for practical deployment

Literature Survey

Author's Name/ Paper Title	Conference/Journal Name and year	Technology/ Design	Results shared by author	What you infer
Singh et al. – Hybrid Deep Learning Threat Intelligence for Malware Detection	Computers & Security, 2025	Hybrid deep learning architecture combining multiple models	Reduced false positives and improved detection robustness	Hybrid architectures outperform single-model cybersecurity systems
Miller & Brown – Explainable Artificial Intelligence for Cyber Threat Intelligence	Open Access Research Journal, 2025	Explainable AI (XAI) techniques	Improved analyst trust and interpretability of AI decisions	Explainability is essential for real-world SOC deployment
Li et al. – Federated Learning for Collaborative Cybersecurity Intelligence	IEEE Security & Privacy, 2024	Federated learning-based distributed security intelligence	Privacy-preserving collaborative learning achieved	Future systems must support distributed and adaptive threat intelligence
				<number> of 12

Detailed Design



Implementation Approach

Goal: A unified threat-detection pipeline that works across **text**, **URLs**, and **network/IDS flows**, while producing **review-friendly evidence**(scores, subtypes, explanations, and HTML summaries).

Modules (end-to-end):

Category A — Text Threats (Spam/Phishing/Scam)

- Inputs: `.eml` / `.txt` messages
- Output: risk score, predicted subtype, explanation (why it was flagged), HTML summary

Category B — URL & Website Threats

- Inputs: URLs from messages or standalone URL lists
- Output: risk score, subtype (e.g., redirect phishing / brand impersonation), top contributing features, HTML summary

Flow/IDS Threat Scoring (KDD-style telemetry)

- Inputs: KDD-like flow CSV (protocol, service, connection statistics)
- Output: per-flow malicious probability + threat categorization (mapped to attack families)

Service Graph + GNN (Service-level anomaly risk)

- Inputs: service $\leftrightarrow$ protocol graph + service statistics
- Output: service-level malicious probability (helps identify risky services, can be fused with flow risk)

Implementation Approach

Categories C & D — Network-Based + Lateral/Internal Attacks (Explainable detector)

- Inputs: KDD-like flow records
- Output: subtype labels (e.g., port scanning, SSH attacks, SMB lateral movement) with explicit explanations + HTML summary.

Tools/Frameworks used (why they fit academic evaluation):

- Python for reproducible experimentation
- pandas / numpy for deterministic feature extraction + batch scoring
- PyTorch for Transformer (flow risk) + GNN (service risk)
- Self-contained HTML reports (single file) for easy review screenshots and audits

Implementation Strategy + Code Evidence (How it works)

Implementation Strategy (why this architecture)

- Hybrid approach (ML + explainable rules):
- ML models (Transformer/GNN) provide robust *risk probabilities* on structured telemetry.
- Explainable detectors (A/B/C/D) provide *attack subtype + evidence* that is understandable by anyone.

Modular + consistent outputs:

- Each detector outputs a consistent schema: `risk`, `pred\_attack`, `category\_group`, `category\_subtype`, `explanation`, `top\_features`.
- Enables a common reporting layer (CSV/JSON/HTML) and easier validation.

False-positive control by design:

- Category C/D uses **eligibility gates** (preconditions) so subtypes only activate when required signals exist.
- Reduces “everything looks malicious” failure modes common in heuristic-only systems.

What this below image proves: the system records *why* a sample was scored high/low (via `contrib` → `top\_features`).

```
def score_linear(features, weights, bias):
    contrib = {}
    z = bias
    for k, w in weights.items():
        x = float(features.get(k, 0.0))
        c = w * x
        contrib[k] = c
        z += c
    return sigmoid(z), contrib
```

```
def eligible(key):
    if key == "ssh_attacks":
        return features.get("is_ssh", 0.0) >= 1.0
    if key == "network_recon":
        return features.get("is_icmp", 0.0) >= 1.0
    if key in {"port_scanning", "service_enumeration"}:
        return features.get("count", 0.0) >= 20 and features.get("diff_srv_rate", 0.0) >= 0.2
    return False

for key, (display, w, b) in C_SUBTYPES.items():
    if eligible(key):
        p, contrib = score_linear(features, w, b)
        best = max(best, (p, display, contrib))
```

What this proves: only relevant subtypes are evaluated, and the highest-risk subtype is selected.

- CSV for quantitative review (risk, predicted label)
- X-TIS style JSON for structured “why” explanations
- Single-file HTML report for panel presentation (risk distribution, top subtypes, top-N examples)

Demonstration

- Demonstration include Working flow, feature validation and Partial Implementation Clarity
 - Working Flow (Input → Process → Output, Screenshot / screen recording)
 - Feature Validation (What feature is demonstrated?, What result is achieved?)
 - Partial Implementation Clarity (What is completed, What is in progress)

Q&A

Thank You